Virtualization:

→ virtualization is a software technology which shows physical resources into virtual resources

→ physical resources first introduced by IBM in 1960's

1. CPU
2. Memory
3. storage
4. Network

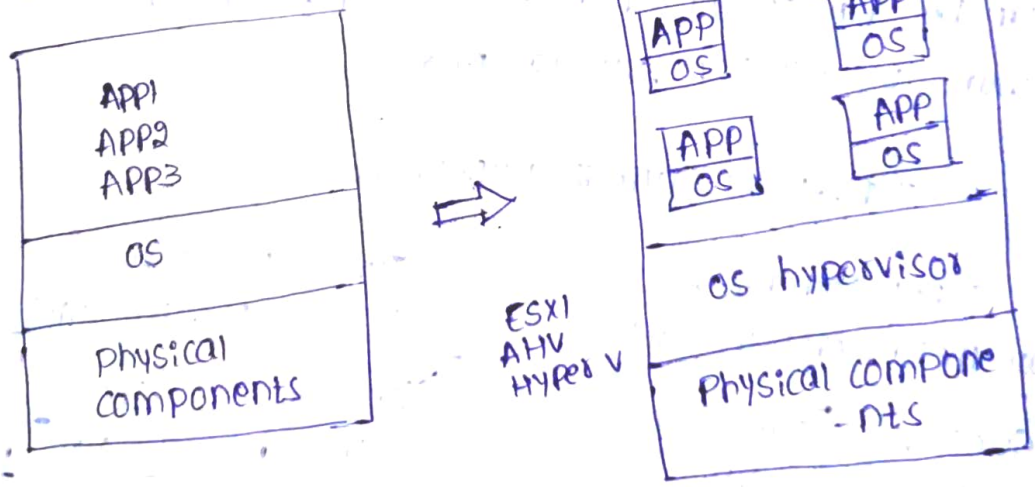
→ It is a technology which makes us to install and run multiple operating systems [Guest operating system] and applications on same physical server at a same-time

Advantages::

1. Effective use of server resources [server consolidation]
2. the maintaince cost of data centers will come down
3. power usage is less
4. No need of more engineers for maintenance

Disadvantages::

1. The major disadvantage of virtualization is if any physical server fails all the virtual machines which are running top of the physical server also will be gone



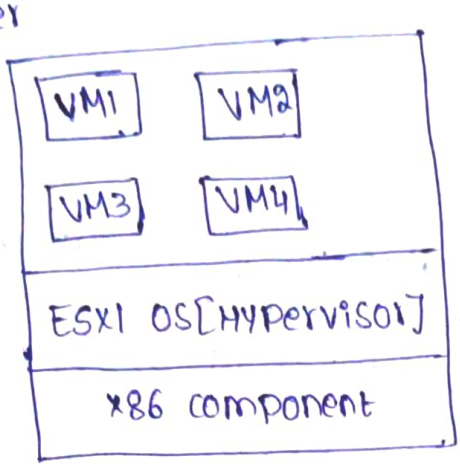
→ Based on the different companies provides the virtualization

- → VMware [Broadcom] > ESXI
- → Microsoft > windows [Hyper V]
- → Redhat [IBM] > Kernal based virtualization
- → Nutanix > AHV
- → Citrix > xen server
- → Oracle > oracle cloud infrastructure

Hypervisor::

→ Hypervisor is a virtualization layer which helps us to run multiple virtual machines on top of single physical server

→ All the virtual machines will share/use the resource of same physical server



there are two types of hypervisor are available

1. Type 1 Hypervisor
2. Type 2 Hypervisor

Type 1 Hypervisor :: → It is more secure, stable and fast

→ It can be run directly on top of the Bare Metal Server [Hardware]

Ex:: VMware, ESXI, Citrix, XenServer, AHV, Nutanix

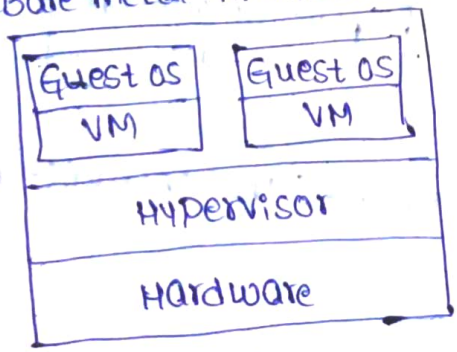
Type II Hypervisor :: → slightly slower because it depends on the host os.

→ It can be run on top of Host operating system

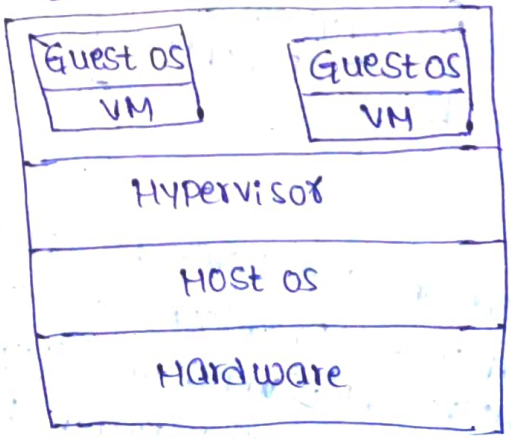
Ex:: VMware workstation, Virtual Box

Type-1 vs Type-2 Hypervisors

Type1 Hypervisor
(Bare metal Architecture)



Type2 Hypervisor
(Hosted Architecture)



Bare metal server::

→ the server which is having no operating system is called as bare metal server

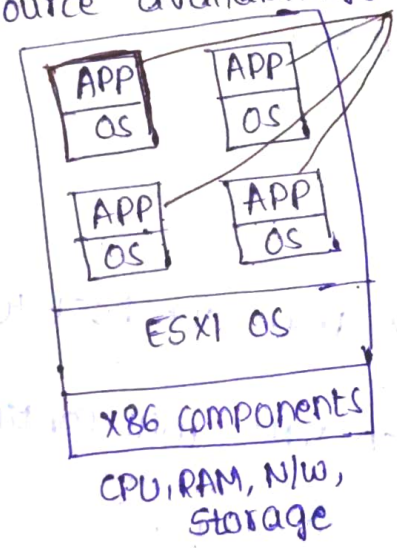
ESXI Host = Physical Server

ESXI::

→ ESXI stands for Elastic Sky X Integrated
→ ESXI is a enterprise operating system from vmware
→ The current version of ESXI is 8.0 [Release 3][Jan 2025]
→ ESXI is a host operating system that we can install on top of servers
→ It is a server operating system

VM [Virtual Machine]:

→ VM is a container where we can install and run Guest OS
→ VM is a equivalent to physical server
→ physical server shares the resources with virtual machine
→ VM is a hardware independent
→ Easy to maintain
→ we can create upto 300 VM's per 1 ESXi Host [Depends on the resource availability]



Components of virtual machine::

1. Guest OS → the OS which we install on VMs
   Ex:: windows, linux
2. virtual CPU [vCPU].
3. virtual storage.
4. virtual switch [vSwitch].
5. virtual memory (RAM).
6. virtual NIC [vNIC].

session-2

20/09/25

Data Store ::

→ the storage space which is formatted with VMFS [VM file system] is called data store

→ Data Store stores virtual machine files, ISO image, configuration files & log files



→ datastore is a storage location where all VM files are stored. It include

→ data stores can be
↳ local datastore :: on host's internal disk
↳ shared datastore :: on external storage like NFS (Network file system), SAN (Storage Area Network)

Storage space space filesystem > VMFS

To manage ESXI ::

1. GUI
2. CLI
3. vsphere / vcentre :: To manage multiple esxi host/clusters

Migration ::

→ Migration is a process of moving virtual machines from one physical server to another physical server with a all minimal down

→ they are two types of Migration

① Hot Migration
② cold Migration

① Hot Migration ::

→ Migrating the powered ON VM from one host to another host

→ In Hot Migration application on VM's will continue to run without any down time → No downtime issue because its already off

② cold Migration ::

→ Migrating the powered off VM from one host to another host

→ cold Migration is prefered when down time is acceptable ~~Hot::~~ zero downtime for the user

↓ the user won't notice any interuption
the VM or application keeps running without pausing or restarting

Adv of Migration ::

① Load Balancing
② Hardware Maintaince

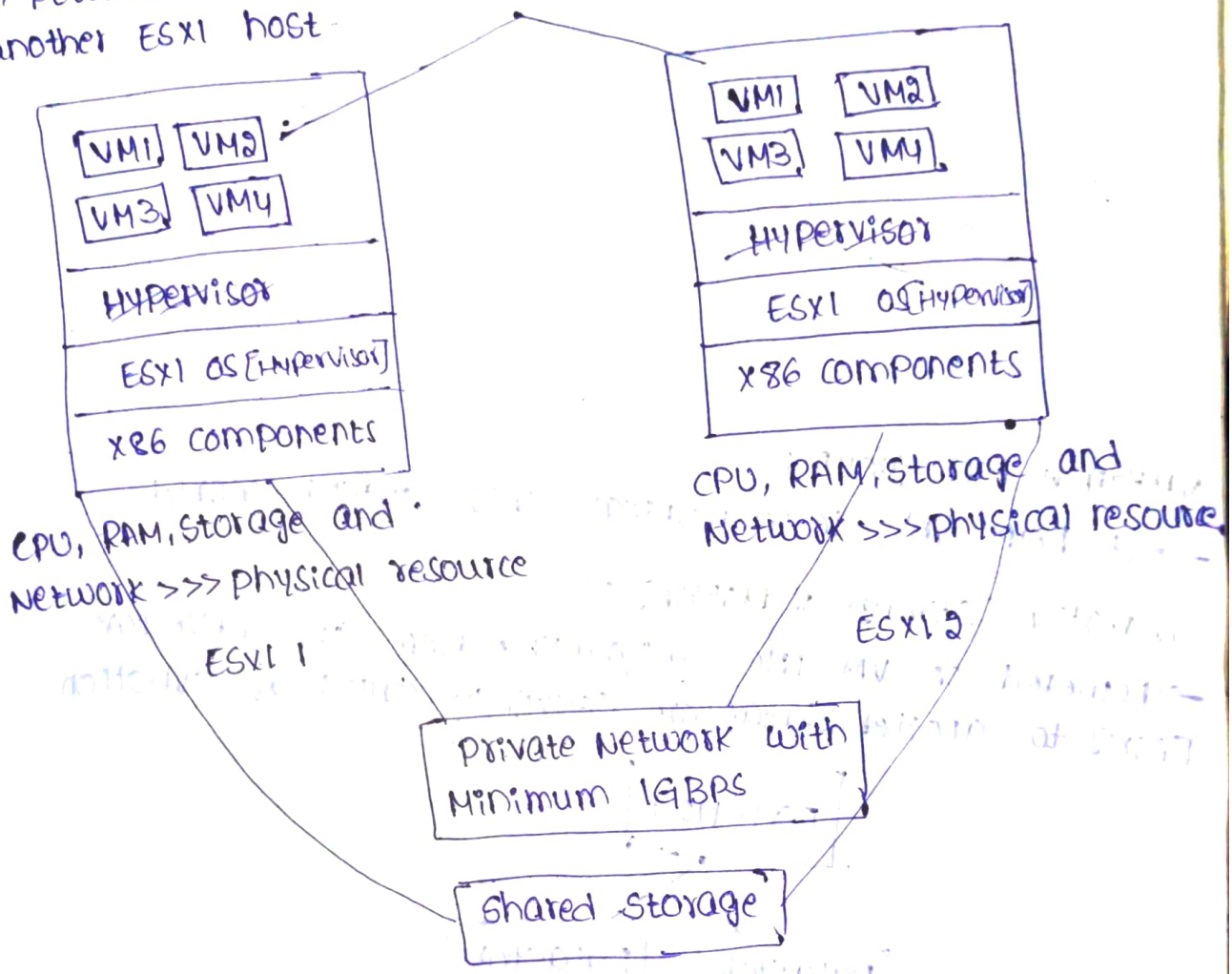
Types of Migration in Virtualization

Features ::

1. VMotion
2. SV Motion
3. XV Motion

VMotion ::

→ Powered on VM's migrating from one ESXI host to the another ESXI host



→ In VMotion CPU memory state will be transfer to the another ESXI host

Requirements :

→ Shared Network and shared storage
→ Same CPU in the source and destination ESXI host
→ VMkernal Port Group must be configured
→ Compatible Hardware at source and destination host
→ Both the host should be in a same network

Why VMotion ::

→ Hardware maintaince
→ To balance the load on VM's ESXI host

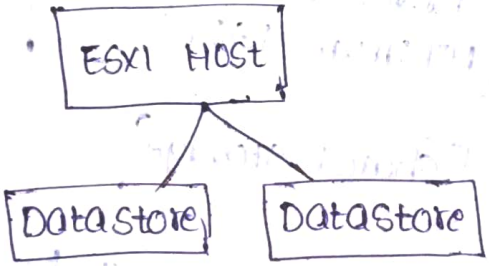
How to perform vMotion ::

VMkernal port group ::

→ It helps VM's to communicate with outside world

SV MOtion [storage v MOtion] ::

→ Powered on VM files migrating from one datastore [ESXI] to another datastore [ESXI] is called as SVMotion



Requirements ::

→ Multiple datastores must be connected to same ESXI host

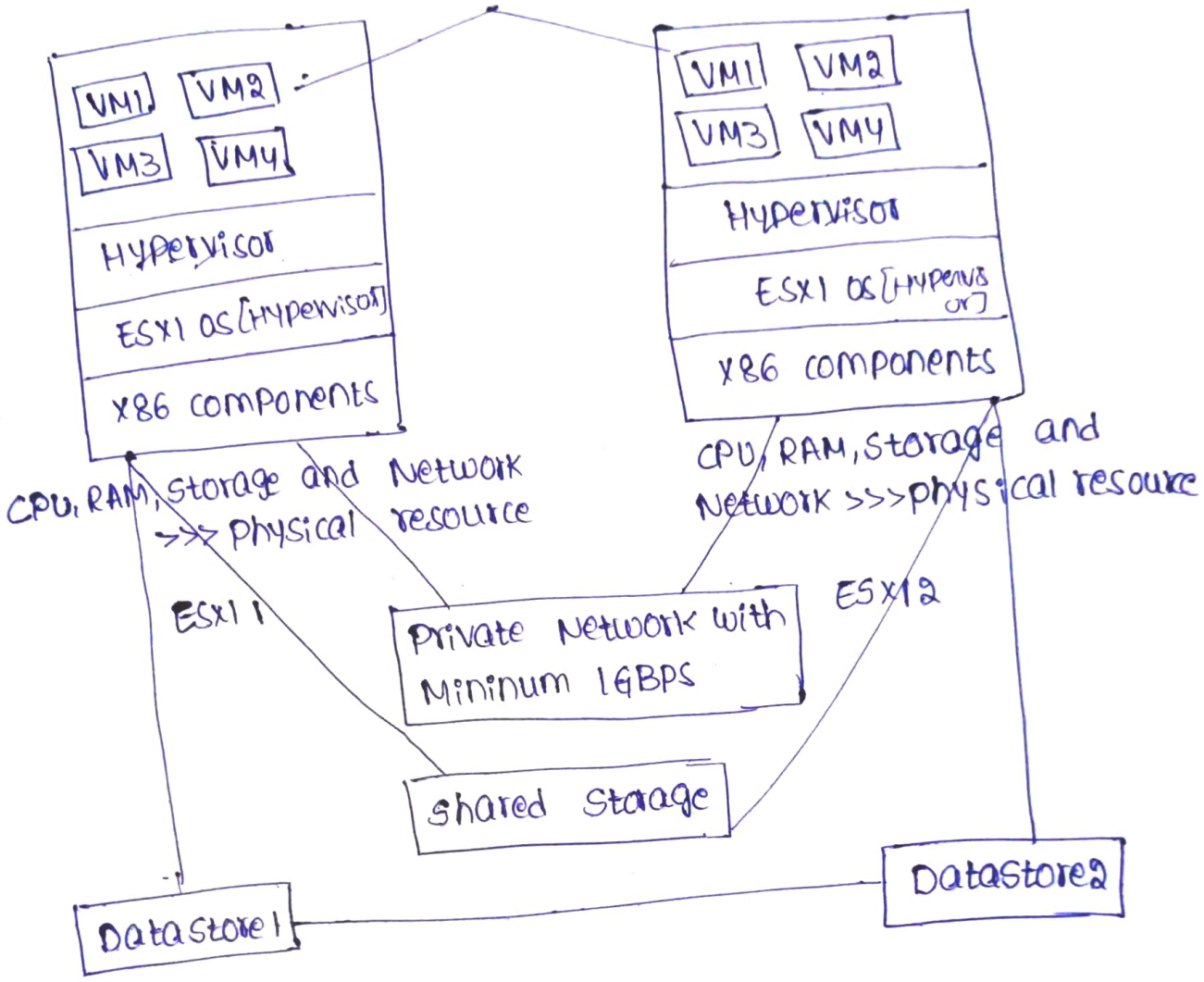
→ Datastore needs enough space for migration

How to perform SV MOtion ::

XV Motion ::

→ Powered on VM's migrating from one ESXI host to another ESXI host and files migrating from one datastore to another data store

XV Motion = VMotion + SV Motion



Requirements ::

→ Shared Network and Shared Storage

→ same CPU in the source and destination ESXI host

→ Vmkernal port Group must be configured

→ Multiple datastores must be connected to same ESXI host

→ Datastore needs enough space for migration

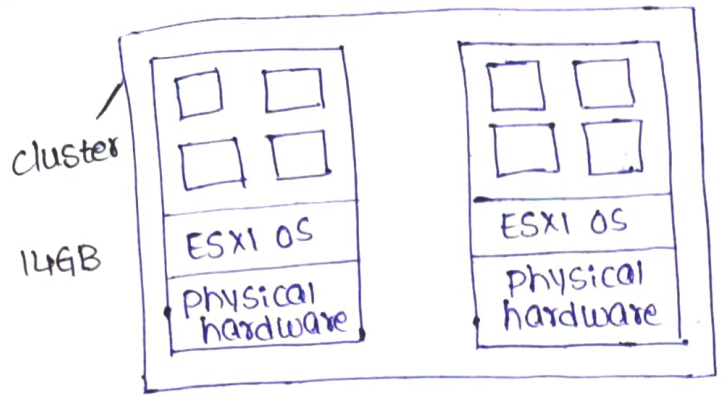
Vcentre / Vsphere ::

It is a application from Vmware used to manage multiple ESXI hosts

→ we need to deploy vcentre VM on any of the hosts

steps to install vcentre ::

cluster:: Group of ESXI hosts
→ when we create clusters by adding multiple ESXI hosts the ESXI host resources will become part of cluster resource



Datastore::
VMFS file system
VM files
.VMX :: Configuration files
.VMdK:: Storage files/ Datastore files → disk files inside datastore
.VMSn:: snapshot file
.vmware.log:: All VM log
vmkernal.log:: All Kernal logs
nvram.log:: SEL log

HA [High availability]::

→ HA stands for high availability
→ whenever the ESXI hosts fails the VM's which are running on ESXI host will be restarted in another ESXI host based on resource availability
① HA should be enabled in cluster level
② when HA is enabled in cluster level, Vcentre installs the HA agent called as FDM [Fault DOMain Manager] on all ESXI hosts within the cluster
③ these HA agents [FDM's] will be in continuous sync with each other and exchanges heart beats
④ when one HA agent stops sending the heart beat other HA agent understands that other ESXI host got failed and starts restarting VMs on working ESXI host
⑤ there will be downtime in HA as VM's need to restarted on other ESXI host

Requirements::

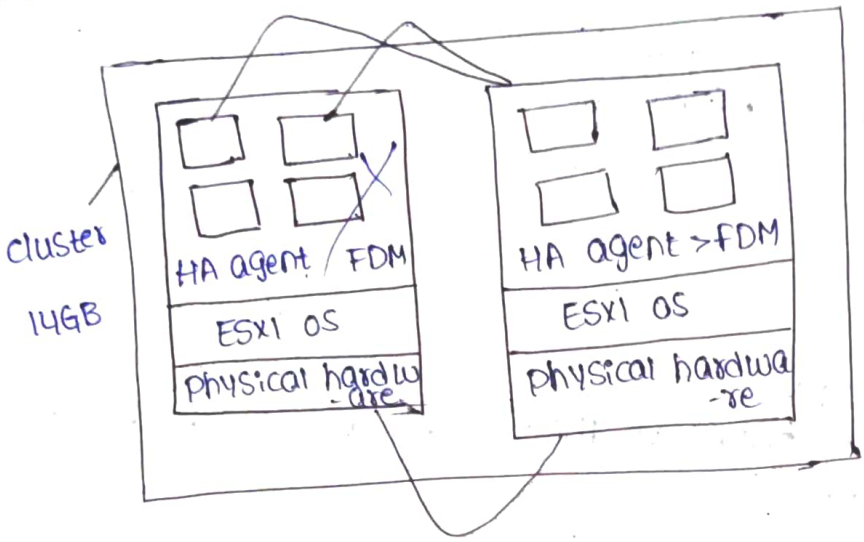
→ there should be enough resources in cluster

↛ HA must be enabled in cluster level

↛ Atleast 2 ESXI hosts must be there in cluster

Disadvantage::

↛ there will be downtime for VM's



FT::[Fault Tolerance]

→ FT stands for fault Tolerance

→ It avoids the complete downtime that is there in HA

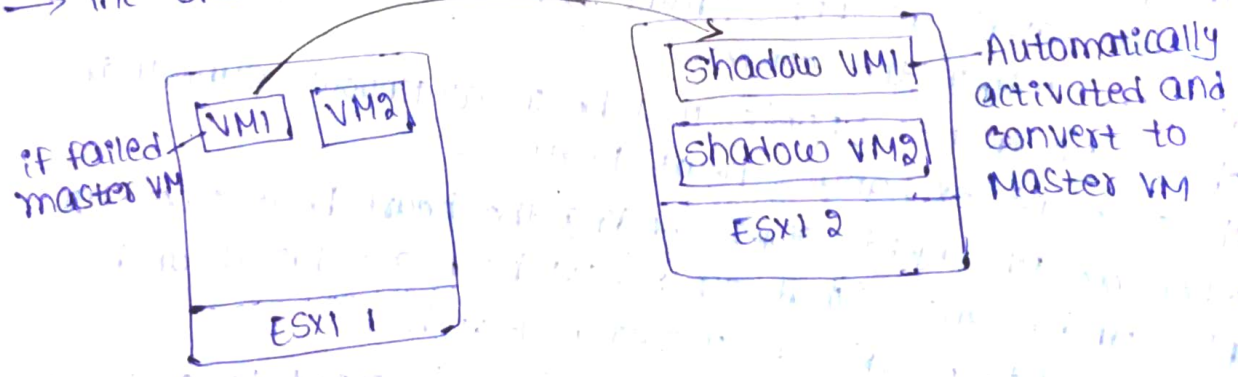
→ FT can be enabled on VM level

→ we need to power-off the VM to enable FT

→ After enabling FT if VM is power-on, the shadow VM will be automatically created on other ESXI host

→ In case of Master VM fails, shadow VM becames master VM and again shadow VM will be created for that particular VM

→ the shadow VM will be in inactive state



How to enabe FT::

① Right click on VM

② select fault Tolerance

③ click on "Turn on fault Tolerance"

Advantages::

→ No downtime in FT

Requirements::

① shared storage
② vm kernal port group must be configured
③ same CPU on source and destination ESXI hosts

DRS [Distributed Resource Scheduler]::

→ DRS stands for distributed Resource Scheduler
→ It is a algorithm applied on group of ESXI hosts [cluster]
→ whenever there is a load in balance on ESXI host, the VM's which are running will be automatically migrated to the other ESXI host. DRS is nothing but automatic VMotion
→ we need to enable DRS on cluster level.

Advantages::

→ Automatic Vmotion
→ used to manage the load on ESXI host

Requirements::

→ shared network and shared storage
→ same CPU in the source and destination ESXI hosts
→ Vmkernal port Group must be [enabled on VM's]
→ Multiple Datastores must be connected to the same ESXI host
→ Datastore needs enough space for migration
→ there should be enough resources in the cluster
→ HA must be enabled in cluster level
→ At least 2 ESXI hosts must be there in cluster
→ same CPU on source and destination ESXI hosts
[All VM's Requirement]

Snapshot::

→ Snapshot is nothing but a point in time copy of virtual machine
→ It captures the state of VM at a specific point in time

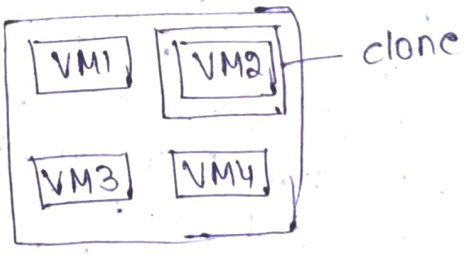
→ the state includes VM power states and VM files
→ we can take upto 32 snapshots per VM
→ snapshot size will be less than the VM size

:.Vmsn
After creating a snapshot, .Vmsn file will be created in data store

we can do following operations on snapshots
① Take snapshot
② Revert snapshot
③ Delete snapshot

How to take snapshot ::
① Login to vcentre / vsphere - ESXI host
② Right click on VM
③ click on snapshot
④ select "Take snapshot"

clone ::
→ It is a exact copy of the VM
→ when we clone a VM it creates duplicate copy of VM
→ clone requires equal amount of diskspace in datastore
→ It is used for backup of data / vm
→ we can take clone on vcentre level only
→ we can take clone when VM is power-on / off state
→ power-on :: 1 clone
→ power-off :: 8 clones

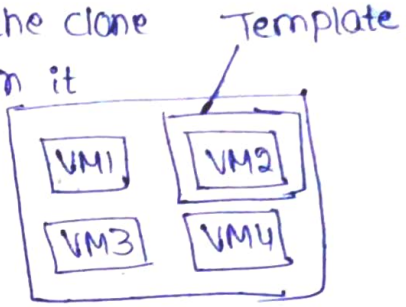


they are 2 types of clone.
① cold clone ::
Taking a clone when the VM is power-off state
② Hot clone ::
Taking a clone when the VM is in power-on state

How to take clone ::
① Login to vcentre / vsphere
② Right click on VM
③ Select clone
④ Give target data store / name of the VM

⑤ select
   ⓐ customize os
   ⓑ customize hardware
   ⓒ power on VM on startup
⑥ click on create

Template ::
→ Template is a golden copy of the virtual machine
→ Template is used to create multiple VM's at a time
→ By using vcentre we can create templates
→ Template takes less time compared to the clone
→ Template is not VM, so we can't power-on it



OVF Template:: open virtualization format
Take template
Deploy template

How to convert VM to template ::
① Login to vcentre
② Right click on VM/cluster
③ convert VM to template
we can deploy a template and create multiple virtual machines

Different types of switch ::
VSwitch :: (Virtual Switch)
→ To connect VM's to outside network
they are two types
① Standard VSwitch :: Establish connection b/w VM's (VM to VM communication)
② Distributed VSwitch :: one ESXI host VM's to communicate with other ESXI host VM's

what are you tested in BMC?
BMC → Out of band
If I do upgrade it got fail?

BMC ⟷ Synchronous / Asynchronous →

use of
Paramiko?
Subprocess?
OS module?

Take i/p from user list of numbers
find second largest number

→ logic in code
dictionary

Class could contain any function
generator
decorator
we need print, create instance and pass values

Project class

Project → platform of storage device / server
solutions testing
vmware

① How to test these products
↳ features of storage products

~~SSD test~~ .
how to test SSD → In this features we will test are storage raid creations, raid levels, raid features, raid states, then we will test features of SSD like wear leaveling, Garbage collection

If we test RAID → If I want more performance which RAID level include and more redundancy which raid level include and if both performance and redundancy which raid level we choose, why we are choosing particular raid level, If R10 is for better performance why you are not going for R0, why you are not going for R5
→ when drive fails in R5 will there be any performance impact, will you be do the read & write operations
→ when raid goes to degraded state we can do read & write operations
→ How to simulate the drive failure

write scenario on degraded state?
→ you remove one physical drive from server that simulate drive failure
→ you do a file reset that simulate drive failure

How to create RAID?
→ we use mdadm, storcli

Removing the drive
How to check raid status → for software → mdadm --detail /dev/mdo
for hardware → storcli /c0 show
How to run IOPS
→ we use fio tool

How to generate 50% read, 50% write operation on fio?
↳ -rw mix = 50%

How you will update firmware? → dell
what if firmware update fail?
How you will make sure that firmware update is successful?

How developer will share the firmware to testers? → GIT JIRA
" you will copied to remote firmware? → SCP
what is the size of firmware file
what is the extension of firmware file → BIOS - .bin
BMC - .hpm
CPLD - .vme
How you will install it? → in drive
If firmware install fails, what you will do?
How you will make sure firmware install is successful?
" " " " " you will installing correct fw?
→ you will check in release notes, developer will give release notes compare with it
→ If mistake in release notes → we need to raise a bug
→ (A) check power cables
→ Reflash it
→ CLI tool
→ Take firmware and install it in another drive (sample)
→ we will check is it is compatability issue
⇒ firmware & product issue are same

you connecting a device drive and drive is not detecting
how you will debug the issue
→ (A) check the physical slot issue
we will swap the drive with another drive
if it is not slot issue means it is hw error

In the server particular drive is not detecting what we need to install → BIOS because it will show all hw components

If drive detecting in BIOS not in os what we will do?
(A) we need to check drivers means update device drivers

Virtualization

How vmotion works?
" you will configure test systems
what is cluster & datastore?
How you will map the datastore

How to install vcentre application ? → VCSA, mount to iso
" " " ESXI ?
" " " Guest OS ?
" " " as from BMC ?

(A) After attaching virtual media it is not recognizing the virtual media what you will do ?
After clicking reboot OS is crashing ?

Testbed

→ How you will connect drive to HBA ?
(A) we will insert HBA in server and connect a cable from HBA to backplance and insert drive to backplane

→ In SAN environment
How we will do zoning
" " " LUN masking } commands & procedure
" " " " mapping

(A) volume name and target ip
command :: map volume --name<> --target_ip=< >

Testbed

→ If we are testing SAN storage system - FOS 3.6.1 fabric operating system
what are components we require :: servers, HBA, Switch, cable, storage Array, JBOD
model numbers we have to aware of :: server, HBA, switch

Recent Bug ?

→ we are doing LUN mapping after mapping LUN we are not able to discover the volume ?
→ if we create raid level we are seeing less performance
→ we are doing performance testing when we ↑sing blocksize IO should be less but we are seeing more

---

Recent developed testcase ?

→ Nume project
↳ we write testcases related to nume namespaces

→ If we are testing HA
test scenarios is
I will turn off one host and I will see vm's are migrating to restarting on another ESXi host or not

---

challenges faced ?

→ when I flashing firmware I faced challenges like just need to take firmware file and deploy it

→ when customer issue comes it is difficult to understand the issue. (says)

→ "I am using big VSAN setup or vmware cluster and I am seeing that one drive is getting disconnected from the cluster

(1) Recreating customer issue is the challenging work

Day to day work ?

(1) Attend daily standup meeting
(2) check emails
(3) " the status of test executions
(4) if any new build we will install
(5) updating the sprint stories / subtask
(6) depend on the story
we will automate the test caseses
create the testcases
configure the testbed
writing the testcases

---

configuring the testbed

→ fresh sever how you will bring up from scratch
(A) insert the Power cables
all the cables
(A) we will insert all required hw like drives, backplane, CPU, ram, BMC, PSU then we will connect req cables like Power cables, HBA cables then Power on server then install os we will bring up server then install required fw on servers

---

Automation framework ?

we use our inhouse framework > pytest based

* folder structure
(1) lib → contains all libraries connect modules. inside Paramiko libraries
testcases
config files
logs → All the logs
conftest.py
↳ setup & cleanup

we have logging module
main libs raid, Power related modules
firmware.py → all firmware modules
raid.py → " raid

json >> Test inputs

version control systems >> Bit bucket
How to clone the repository
add the files
commiting the changes
creating the pull request

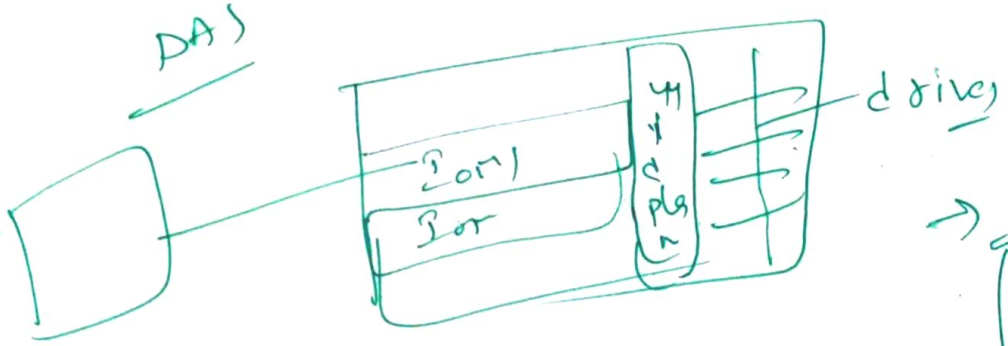
Recently Automated testcases

(P1) → platform validation

VSP E590H

Storage Array

DAE



→ platform

BMC
BIOS
CPLD
drives
Midplane
IO module

2U 24

24 SSD (2.5 inches)

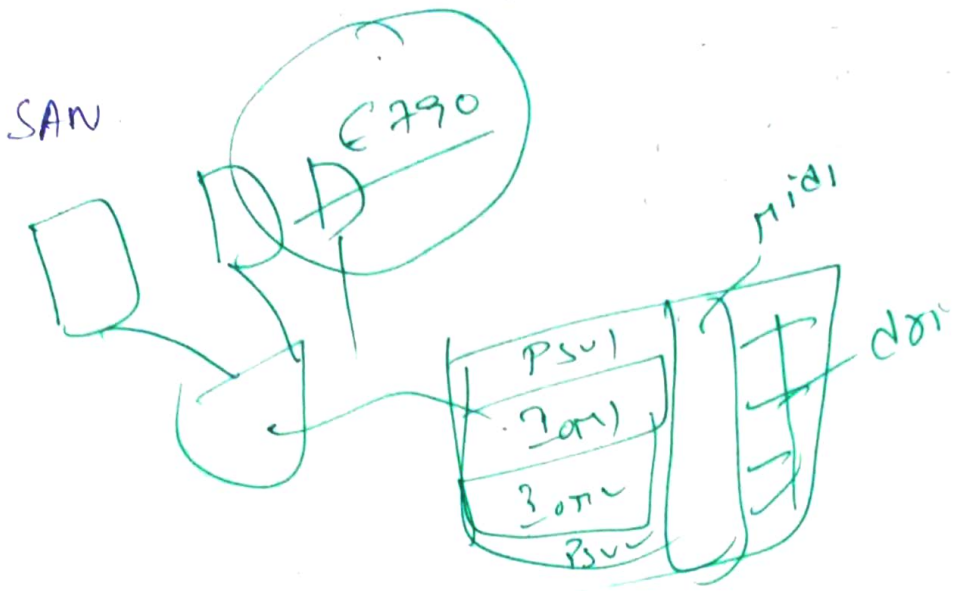
upto

30TB NVME
30TB SAS SSD
18TB HDD

Max → IO Expansion Enclosure ✓ It supports
↳ JBOD's

Max drives
55.2

→ F/W upgrade
→ Reliability Test
→ drives



FC
ISCSI
Numeof

FC → 32 Gbps
ISCSI → 10 Gbps

Python 44
↓
Git

SAN features
VMware features

→ LUN | RAID | volume | map | mask | zones

↳ performance testing
↳ VMware features (vSAN)
↳ f/w upgrade | dam
→ stability
→ protocol levels
tests

Replication
copy data

storcli /c0 add VD type=raid0 drives=252:0 (252 → enclosure id, 0 → slot number)

Non-disruptive Migration
↳ hot Migration

Midplane → acts a connector b/w controller and drives
I/O module → connects Storage → Server
JBOD's → 10 JBOD's → 552 drives

If firmware update fails ::
logs → CLI method → ipmitool sel list
Linux → dmesg
cat /var/log/messages

Disk/RAID logs → storcli /c0 show
storcli /c0 /eall /sall show

SAS controller logs → sas3flash -list

what we have to check in logs :: Error messages
warning alerts
failed components

If what if you can't fix the issue?
① Analyze : check logs, identify issue
② Try basic fixes : Restart, reseat, retry update
③ validate ; check if issue resolved
④ escalate → with Proper issue description
system details
firmware version
logs

you connecting a device drive and drive is not detecting how u will debug the issue?

→ ① check physical slot
② swap drive with another drive
③ If it is not slot issue means it is hw issue

In server particular drive is not detecting wht we need to install?

(A) Bios because it will show all hw components

If drive detecting in bios not in os what we will do?

(A) check drivers means update device drivers

↓

means it allow computer os to storage hardware

VSP E590H → It is an mid-range enterprise storage system used in datacenters, cloud environment, large enterprise application
→ NVMe + SAS
→ Its internal storage is upto 10.62 PB
It supports NVMe SSD, SAS SSD and HDD
It can store huge data
→ Its IOPS is upto 4 million
Bandwidth is 22GB/s
means fast read/write operations
→ Its data reduction up to 7:1 efficiency
(compression + deduplications)
saves storage space
→ No downtime
→ It supports SSD drives
external capacity up to 144 PB
→ It supports protocols like FC & ISCSI
used to connect servers to storage
→ It supports RAID like RAID 5, RAID 6, RAID 1
→ Its features include RAID, deduplication, CLI, remote copy, hot migration, Replication

common errors ::

① version mismatch :: firmware not compatible with hardware
Symptoms :: update fails immediately
"unsupported firmware" error
fix :: check release notes
download correct version
verify model number

② power failure :: system shutdown during update
Impact :: firmware corruption
System may not boot
fix :: After power failure BMC was not accessible. we used recovery mode to reflash firmware
↓
By using USB, (or) CLI

USB Recovery :: → .bin / .img file
→ then copy to usb
→ Connect usb to server
→ ON the system
Boot f12
→ select Recovery option
→ select usb and flash the firmware

(if BMC not accessible after update means we have to use usb, CLI)

CLI Recovery :: ipmitool hpm upgrade firmware.bin

③ Device not detected :: Drive / module missing after update
Fix :: After drive firmware update disk was not detected we need to reseated drive and rolled back firmware
↓
reinstalling a previous stable version of a device os

If firmware update fails ?
→ ① check logs → ipmitool sel list, dmesg, storcli
→ ③ Reflash
→ ② check compatibility → we need to see in release notes like model, firmware version
→ ④ use recovery mode

How to confirm firmware update success ?
→ version check
→ No errors in logs
→ All components working

If debugging drive not detected ?
→ check physical slot
→ Reseat drive
→ swap with another slot
→ check in BIOS
→ check drivers

Sai mohan Sutherland
→ day to day responsibilities
→ testing responsibilities
→ hardware (or) Virtual
→ os installation u perform
→ bootable usb how to use it ?
→ once installation complete what u will check first ?
→ during os installation system doesn't boot how to troubleshoot it ?
→ SDLC, STLC
→ How u raise a bug
→ in a sprint how many bugs u raised
→ BUG life cycle
→ how u design test case by using os installation ④ based on RAID levels

→ Particular hardware ④ os installation in RAID 5
→ in windows server where u check logs which drive
→ in windows command what command u used for system validation
→ disk, services, which command u used in Linux

SSD validation :: It means verifying that the drive is properly detected, healthy, performing correctly and working reliability under diff conditions like RAID and workload testing

① Detection check :: storcli /c0/eall/sall show
↳ if it is visible ok
↳ Not visible "issue

② Health check :: smartctl -a /dev/sdX
↳ It will check wear level, errors, temperature

③ Read/write testing :: fio --name=test --rw=readwrite --size=1G
↳ No error : valid

NVMe ::

① Detection validation :: nvme list
↳ device name, size, model → if visible ok

② Health check :: nvme smart-log /dev/nvme0n1
↳ it check temp, media errors, % used → if no errors → healthy

③ Read/write testing ::
fio --name=nvme_test --filename=/dev/nvme0n1 --rw=randrw --size=2G --runtime=60
↳ it check IOPS, Latency, throughput

④ RAID compatability :: Add NVMe to RAID
Remove NVMe
↳ it expects :: RAID degraded, Rebuild works

HMO-7 → Automatic LUN recognition
HMO 73 → windows 78 → Active-Active path

→ self
→ manual testing how many experience
→ regressing testing
→ Smoke testing
→ Automation which tools u are using
→ list & tuple
→ Tell me what u validate in automation & manual

→ what is fullform of RAID
→ Sathyaland
→ after firmware update what u will check
→ how u use Powershell - yes dashboard
→ u have test execution log file how to build log file
how u will read log file in Python (A) using filehandling
how u convert to Html report

→ how u validate means how u debug if it fails
→ Automation scripts locally how to push code in repository
→ In ur Project major impact
→ In ur laptop u have to connect usb drive
→ M
testing to connect laptop ; it won't connect how you will debug

If NVme drive not detected ::

Step-① : check PCIe detection

lspci | grep -i nvme

↓

If it is not dected :: Hardware issue

→ check connection
→ Try diff slot
→ verify BIOS setlings
→ Test in another system

Step②:

If it is detected :: check os detection

nvme list → cmd

If it is not detected in OS :: Software/driver proble

→ check logs (dmesg)
→ Reload NVme driver
→ update Kernal
→ Rollback firmware

If it detected ::

drive working → check performance

Step③ ::

If performance issue :: error/slow

check file systen
run diagnostics
Turn I/o

Step④ ::

sometimes detected, sometimes not

↓

It is PCIe instability

↓

we need to fix like → change slot
update BIOS

Bios failure ::

Step① :: check system power ON?
↓ NO
→ check power supply cables
↓ yes
→ Nxt step

Step② :: Any beep codes?
↓ yes
→ Hardware issue
→ Diagnose using beep code
→ NO
→ Nxt step

Step③ :: clear CMOS
→ Reset Bios settings

Step④ :: Reseat components
→ RAM
→ GPU

Step⑤ :: Test minimal hardware
→ only CPU + RAM

Step⑥ :: Bios screen Appears
↓ yes
→ update Bios / reinstall OS
↓ NO
→ replace / repair

BMC failure :: After flashing Bmc it is not accessible How do u check it

Step① :: is BMC accessible
↓ NO
→ check (IP, cable, Switch)

Step② :: Reset BMC
→ Try reboot / reset → still not working → Reflash firmware
→ Power cycle system
→ Replace motherboard

ipmitool mc reset cold
" sel list

If it is accessible → check IPMI commands

Step③ :: IPMI working
↓ NO
→ Restart services
→ Reload drivers
→ update firmware
↓ yes
→ check logs & sensors

Step④ :: logs / sensors
→ fix sensor issues → check errors

Firmware updates ::

Nvme ::

① firmware download
② " activate
③ " commit

Step-① : Download firmware

nvme fw-download /dev/nvme0 --fw=/path/to/fw.bin

Step② : commit & Activate (on reboot)

nvme fw-commit /dev/nvme0 --slot=1 --action=1

Step③ : Reboot system

reboot

Slot=1 → primary slot
action=0 → Replace image
action=1 → Activate at nxt reset
action=2 → Activate immediately
action=3 → Activate after reset
nvme0 ↔ It represents nvme controller

BIOS ::

1. from OS ::

duu --fw_version -d <device_name> → check fw version

To update firmware

duu --fw_update --device <device_name> --fwpath <firmware_file>

2. By using external tools ::

dediprog
connect cables from dediprog to BIOS
1 cable from dediprog to BIOS pin
1 cable from USB port
discover BIOS in dediprog software
select firmware update
select firmware file
click on update
check firmware version

3. from bmc ::

ipmitool -I lanplus -H <BMC_IP> -U <user> -P <password>
hpm upgrade <firmware_file>
↓ hardware platform management

sas/sata firmware ::

① using RAID controller tool (storcli) ::

→ copy firmware to server
→ Identify drives
storcli64 /c0 /eall /sall show → c0 → controller 0
ex → enclosure
sx → slot
→ start firmware update
for single drive :: storcli64 /c0 /ex /sx download file=firmware.bin
for all drives :: storcli64 /c0 /eall /sall " " "
→ monitor status
storcli64 /c0 show
→ Reboot system
reboot

hardware raid create
storcli /c0 add vd type=raid1 drives=252:0
252 : enclosure ID
0 : slot number
VD → virtual drive
raid1 → raid level
/c0 → controller 0

② using sg3-utils ::

→ Identify device
lsblk
→ check firmware
sg_inq /dev/sdb
→ update firmware
sg_write_buffer --mode=5 --in=firmware.bin /dev/sdb
→ verify firmware after update
sg_inq /dev/sdb

modes :: 0 → write data
2 → write and save
5 → download firmware

Drives ::

hdparm --fwdownload {fw_paths} --yes-i-know-what-i-am-doing-please-destroy-my-drive <drive_name>
↓ JxtGOW4Q2023.bin
↓ /dev/sdb

Recent bug ::

Bug 1 :: Nvme drive was not detected in os
we have to check PCIe detection using lspci
It was detected at hw level
checked os using nvme list → not visible
fix : check logs (dmesg)
Reload nvme driver
After reload → drive detected (modprobe nvme)

Bug 2 :: BMC was not accessible via IP
checked IP, switch, cable
Network was fine
Tried BMC reset (ipmitool mc reset cold)
fix : reflashed BMC firmware
performed power cycle

Bug 3 :: RAID went into degraded state
checked using storcli (storcli64 /c0 show
storcli64 /c0 /eall /sall show)

Identified failed drive
fix : replaced drive
RAID rebuild started

→ if firmware update fails
→ version mismatch
→ power failure (system shutdown during update)
→ device not detected
→ if BIOS fail
→ why R10 is faster than R5
→ what happens if one disk fails in R5
→ Performance drops during degraded mode
→ what happens if rebuild does not start automatically
→ Lun created & mapped but Linux host does not detect the disk
→ RAID rebuild doesnot started automatically
→ Performance issue after firmware upgrade

Zoning → LUN creation → Host group creation → LUN mapping
→ LUN masking → Host scan → multipath → snapshot

① Zoning : connect Host HBA ↔ Storage port

This is cisco method ::

→ create zone ::

```
conf t
zone name ZONE1 vsan 10
member pwwn 10:00:00:00:00:00:01   # Host HBA
   "        "   50:  "  "  "  ":02   # Storage port
exit
```

→ Add to zoneset ::

```
zoneset name ZS1 vsan 10
member ZONE1
exit
```

→ Activate zoneset ::

```
zoneset activate name ZS1 vsan 10
```

(or)

This is Brocade method ::

Switch → FOS 3.6.1 ↓ Fabric operating System

→ create Alias ::

```
alicreate "HOST1","10:00:..."
alicreate "STORAGE1","50:00:..."
```

→ create zone ::

```
zonecreate "ZONE1","HOST1;STORAGE1"
```

→ create config ::

```
cfgcreate "CFG1","ZONE1"
```

→ Enable config ::

```
cfgenable "CFG1"
```

→ save ::

```
cfgsave
```

② LUN creation (on Storage) :: it create storage volume

Steps :: create RAID group
" volume (LUN)

command :: Volume create -name VOL1 --raid_group RG1 --size 500g
Create volume size=100GB POOL=POOL1

⑧ HOST group creation ::

add hostgroup HG1 PORT CL1-A → these will connect to SAN switch
↳ storage port

④ LUN mapping :: assign LUN to storage port

map volume VOL1 PORT CL1-A lun 10

⑤ LUN masking :: allow only specific host

add host HG1 WWN 10:00:00:00:00:01 → wwn addition
add lun HG1 VOL1 lun_id=10 → LUN assignment

⑥ HOST side ::

Rescan disks ::

echo "- - -" > /sys/class/scsi_host/host0/scan

check LUN is visible by using this command:

lsblk

multipath validation :: → multipath creation: ① install package

check LUN is visible → yum install device-mapper -multipath
apt install multipath-tools

lsblk

② Enable multipath service

check multipath service systemctl enable multipathd
systemctl status multipathd " start "

③ Generate multipath config

Start service mpathconf --enable

systemctl start multipathd
" enable "

⑦ Rescan

check multipath devices

multipath -ll

echo "- - -" > /sys/class/scsi_host/host0/scan

⑧ Start multipath discovery

verify OS multipath -v2

lsblk

Snapshot validation :: (created on Storage) ⑥ verify multipath -ll

Rescan to detect snapshot LUN

echo "- - -" > /sys/class/scsi_host/host0/scan

check new device

lsblk

check file system : mkfs.ext4 /dev/mpathb

blkid

mount snapshot

mkdir /mnt/snap

mount /dev/mpathb /mnt/snap

Validate data

cd /mnt/snap

ls

unmount (after check)

umount /mnt/snap

→ check dmesg if device not visible
→ multipath must be started before validation
→ snapshot LUN must be mapped and masked to host

How to generate test reports
How to run multiple tests in parallel :: By using pytest -xdist
conftest.py :: Before testcase execution conftest be called
↳ we can use it as a setup and ti·function
can we create multiple fixtures inside conftest

---

create project → create repositord → copy the url in gitbash → ls → cd repository name (Ex: repo_d) → git branch Anyname (Ex: student) → git checkout student → ~~git log (it is to see when and email. and all) → git add~~

---

create project → create repository → copy the url in gitbash (clone) → ls → cd repository name (Ex: cd repo_d) → git branch Anyname (Ex: git branch student) → git checkout student → touch file1.py → ls → git log (it will show details) → git add file1.py → git commit -m "A new file created" → git config --global user.email "dhanujabandaru02@gmail.com" → git config -global user.name "dhanuja" → git push --set-upstream origin student → copy the url in gitbash to chrome → It will open pull request page → click on create pull request → Now come to gitbash → Enter git diff → vi file1.py → press i → print ("Any msg") → click esc → :wq! → git diff → git add file1.py → git commit -m "Added new line" → git push → refresh chrome page → merge

RAID:: 0, 1, 5, 6, 10

Why R10 is faster than R5?

(A) Because R10 follows stripping across mirroring without parity calculation, while R5 uses parity calculation which adds write overhead

→ performance dropped after RAID creation
Because background rebuild running

What happens if one disk fails in RAID 5?

(A) RAID5 supports one disk failure. when one disk fails, RAID enters degraded state. but data remains accessible using parity information. I/o continuous without downtime. After replacing the failed disk rebuild starts automatically and RAID returns to nrml state

- Why performance drops during degraded mode?

(A) Because the controller must calculate missing data using parity from remaining disks so it reduce performance

- what happens if Rebuild does not start Automatically?

(A) check whether replacement disk is properly detected and verify size and type compatibility then I check RAID health, controller logs. If controller issue exists I collect logs and proceed with Root cause Analysis

Zoning::

- Host is unable to see Storage after LUN mapping?

(A) first I check zoning because if zoning fails host cannot reach storage. I verify correct WWPN, active zoneset, switch port status.

we created raid level but we are seeing less performance
(1) check RAID health → storcli /c0 show (or) mdadm --detail /dev/md0 (if "optimal"
(2) " RAID rebuild → if RAID rebuild performance will lower
(3) " disk health → smartctl -a /dev/sda
(4) verify RAID level → R5 has parity so write performance low
(5) check sys Resource usage → check cpu and memory utilization → top
(6) check I/O performance → iostat (or) fio
(7) check logs → dmesg → look for disk error, RAID error, controller error

Bugs raised ::

① LUN mapped but host cannot discover volume

(A) LUN created and mapped successfully but Linux host could not detect the disk

How to validate :: checked zoning, WWPN, mapping, masking, host rescan done, found wrong HMO (Host mode option) ↓ used to address specific os behaviors

→ Appliped correct Host mode option
→ Host discovered disk successfully

② RAID rebuild not starting Automatically

(A) After drive failure RAID became degraded but rebuild did not start

How to validate :: checked spare disk, disk type, rebuild policy

→ Added correct spare drive
→ Rebuid started automatically

③ performance issue after firmware upgrade

(A) IOPs dropped after upgrade

How to validate :: checked latency, cache status, compared old performance logs Check raid health using storcli, verify logs, multipath status using multipath -ll so write cache disabled after firmware upgrade causing performance degradation : After enabling return to normal

→ Re-enabled write cache settings
→ Performance returned normal

④ controller failover caused path loss

(A) During failover testing, host lost all paths

How to validate :: corrected multipath configuration ↓ to view current active path devices and their paths

→ failover worked without downtime

Recently Automated test cases ::

① Drive health validation

(A) checks :: all drives detected, failed drives, blocked drives, warning status

② Port status validation

(A) checks :: all fc ports UP ↓ physical connections, logical connections, login successful, speed correct

③ LUN validation
Ⓐ checks : mapped correctly, visible to correct host

④ RAID status validation
Ⓐ checks : RAID nrml, degraded, rebuild progress

Developed Testcase ::

Validate RAID rebuild

Steps :: create RAID5
verify RAID nrml
Remove one disk
verify degraded state
check spare takeover
verify rebuild starts
replace failed drive
verify raid returns nrml

Result : Rebuild should completely succeful!

Test scenarious

① validate LUN creation and host discovery
② " RAID rebuild after disk failure
③ " firmware upgrade without path loss
④ " controller failover during active IO
⑤ " performance under heavy IO load
⑥ " psu failover using dual power supply

⑤Ⓐ During heavy IO testing, latency used too much and IOPS dropped unexpectedly
checks :: RAID health status, rebuild actively running or not, controller CPU usage, cache status
→ RAID rebuild was running in the background which cause performance degradation
→ After rebuild completed latency became normal and IOPS improved

⑥Ⓐ while testing psu failover after removing one psu, system showed power warning alerts
checks :: second psu status, power cable connection, power source availability, psu health status

→ second PSU was connected but power source was off
→ After removing one PSU, storage continued running normally without shutdown

---

DIMM not detected after system boot
① check memory from OS :: free -h
dmidecode -t memory

② check kernal msgs :: dmesg grep -i memory

reseat DIMM
swap slots

---

echo "---" > /sys/class/scsi_host/host0/scan
↳ virtual filesystem created by linux kernal
↳ storage controller
↳ scsi storage controller information

channel, target, LUN
All All

→ Ask linux kernal to rescan all scsi devices connected to storage controller host0

Madhavi Sutherland
→ diff b/w list & tuple
→ features
→ BMC
→ Powershell in server states, in windows

madhavi UST
→ self
→ Bios, features of BIOS
→ BIOS Booting process (BOS, MBR, CPUs ...)
→ If any server in your server
→ what type of server currently you are using
→ is it single (or) multi socket
→ memory models u r working
→ if server fails boot to OS how u will rectify
→ what is smbios
→ how to get chassis's info by using smbios
→ diff b/w smbios & dmidecode
↓ linux utility
→ I want chassis info, Platform info
→ few ipmitool commands ? inband & outband

→ redfish method to update OS (BMC, dedi prog)
→ what is BMC
→ familiar with windows & linux
→ how u will get CPU info
→ major feature to verify in CPU info
↳ (A) socket count, thread count
→ diff b/w core & thread
→ multithread use do it?
→ PCI Architecture
PCI Basic precautions to perform hardware
→ How to check width & speed in PCI
→ y connected one storage (samsung) Gen 5 what did check in it?
→ Gen 5 speed, width
→ u Performing RAID how u will create inside OS (or) using BMC
→ RAID 0 & RAID 5 diff
→ BUG life cycle
→ Server BIOS QA role
roles & responsibilities

modules:: It is just a python file (.py) that contain functions, classes and variables

→ Group of functions is called modules

Why module? → To reuse code

Types: ① user defined :: test1.py
② python inbuilt :: os, sys, time, numpy
③ user installed :: pytest, paramiko

os:: used to execute system commands in linux / windows

os.system() : It will return status code → "0" - success, "non-zero" - error
os.popen() : " " " command o/p
os.getcwd() : It represents current working directory (cwd)
os.chdir() : " " change directory
os.listdir() : To list files & folders
os.mkdir() : To create new directory
os.rmdir() : To remove directory
~~os.curdir~~
os.curdir :: '.' :: current directory
os.environ :: To get system environ variables
os.path.exists :: check if file exists or not
os.path.abspath :: To get full path converts relative path → absolute path. It gives the full path as it appears
os.path.realpath :: ~~to get basename of path~~
↳ follows the symbolic link and gives the true destination.

Sys:: used to interact with system and get / change system configuration

sys.argv :: used to read command line arguments

Ex:: import sys
print(sys.argv)
print(sys.argv[0])
print(sys.argv[1])

o/p: python os1.py dhanuja
['os1.py', 'dhanuja']
os1.py
dhanuja

~~sys.po~~
sys.exit() :: stops the prgm.

Ex:: import sys
print("start")
sys.exit()
print("stop") → It will not run

sys.path :: python will search for user defined module
sys.version :: python version (detailed) 3.10.11 (tags ~~~~)
sys.getsizeof() :: to get size of a variable
sys.winver :: python version 3.10
sys.platform :: platform details win32

argparse :: used to read command line arguments

```
import argparse
parser1 = argparse.ArgumentParser()
parser1.add_argument("name")
args = parser1.parse_args()
print(f"Hello {args.name}")
```

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("--age", type=int, help="Enter age")
args = parser.parse_args()
print(args.age)
```

o/p: python os1.py --age=21
21

logging → To push msgs to log file :: used to track and record prgm execution details such as debugging, information, warnings and errors, and critical

```
import logging
logging.basicConfig(
        filename="modlog1.log",
        filemode="w",
        level=logging.INFO,
        format="%(asctime)s - %(levelname)s - %(message)s"

)
def fun1():
    logging.info("this is my desktop")
fun1()
```

o/p: you will see in modlog1.log file
2026-05-01 13:23:17,064 - INFO - this is my desktop

os.walk :: It is used to walk through

threading: It allows a prgm to execute multiple threads parallely

```
import threading
import time
def fun1():
    for i in range(10):
        print(f"task {i}")
        time.sleep(2)

def fun2():
    for i in range(10):
        print(f"task {i}")
        time.sleep(2)

thread1 = threading.Thread(target=fun1())
thread2 = threading.Thread(target=fun2())

thread1.start()
thread2.start()

thread1.join()
thread2.join()
```

o/p: task 0, task 1, task 2, 3, 4, 5, 6, 7, 8, 9, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

→ for every task it will wait for 2 sec

paramiko: used to communicate with remote servers with SSH protocol:

→ we can execute cmds on the remote servers and get the response using paramiko

```
import paramiko

ip = "192.168.0.100"
username = "winteck"
password = "winteck@2026"
port = 22

def exec_cmd(command):
    client = paramiko.SSHClient()
    client.
```

```
def exec_cmd(command):
    obj1 = paramiko.SSHClient()
    obj1.set_missing_host_key_policy(Paramiko.AutoAddPolicy())

    obj1.connect(hostname=ip, username=username, password=password, port=port)

    stdin, stdout, stderr = obj1.exec_command(command)
```

```
    output = stdout.read().decode()
    obj1.close()
    return output

c = exec_cmd("lsblk")
print(c)
```

Subprocess::

```
import subprocess
subprocess.run(["cmd", "/c", "dir"])

process = subprocess.Popen(["cmd", "/c", "dir"], stdout =
                                        subprocess.PIPE)

output = process.communicate()[0]
print(output.decode())
subprocess.call(["cmd", "/c", "dir"])
```

/c → run and exit ; /K → run and stay open

["cmd", "/c", "dir"] → means open cmd → execute dir → close cmd

Subprocess.run → runs a cmd, waits, and return result

Subprocess.Popen → starts a process and gives full control over it

Subprocess.call → Runs cmd and returns only exit status

stdin → input , stdout → normal o/p , stderr → error o/p

process.communicate → wait for process to finish

[0] → first value (stdout) , [1] → second value (stderr)

→ Subprocess.**PIPE** → used to capture data the o/p

output.decode() → converts byte to string

Paramiko.SSHClient() → creating SSH object means used to connect to a remote server and run cmds

AutoAddPolicy() → Accepts unknown server automatically

connect(IP, username, password, Port) → connect server using these login credentials

Decorator :: Decorator is a special type of function in Python
→ we can decorate function with decorator name
→ Decorator is a function that modifies another function without changing its code

Generator :: Generator is a special type of function in Python. used to generate sequence
→ It returns value one at a time
→ Generator function will return generator object
→ If we want to access individual iterable we can use next obj / function
→ we use "yield" instead of "return"
→ yield will capture the state of object

return	yield
→ It is normal function	→ It is generator functions
→ sends back a value from function to the caller	→ sends a value one at a time
→ It ends the function	→ It pauses the function
→ function will not continue after return	→ Remember its state and continue later

class :: class is a blueprint to create object
→ we can define class by using keyword "class"
→ class contains attributes and methods

object :: It is an instance of a class
By using object we can access properties of a class

self :: It ~~return~~ refers the current object
→ In the place of self we can use any other name as first argument and it is mandatory
→ self is mandatory in instance method

__init__ :: __init__ is a constructor and special method in python used to initialize all the attributes
→ __init__ is a instance method and first argument is "self"

class variable :: It is defined inside the class and outside the method

Instance variable :: It is defined inside the instance method

Inheritance :: Inheriting the properties of parent class to child class
Single Inheritance :: Inheriting the properties of one parent class to child class
multi-level Inheritance :: Inheriting the properties of parent class to child class and child class to another child class
multiple Inheritance :: Inheriting the properties of multiple parent class to child class
super() :: used to inherit parent class constructor in child class and it is mainly used in multi-level inheritence
MRO :: (Method Resoultion order) - It defines how python will look for method incase of inheritance
Public :: It will available across the class. we can access/ modify it with outside the class also
protected :: It can't be accessed outside the class
It is defined with "_" (single underscore)
private :: It can't be accessed directly outside the class. It is accessible within parent class only.
It is defined with "__" (double underscore)
It can be accessed outside the class by using "name mangling".
Instance method :: By using instance method we can access class level attributes and instance attributes
class method :: By using class method we can access class level attributes. first argument should be "cls"
→ It is decorated with @classmethod
→ we can't access instance attribute with "cls"
Static method :: If we are not going to access class (or) instance attributes (or) methods we will make method as Static method
→ It is decorated with @staticmethod
→ It need not require any arguments
Encapsulation :: Binding data and methods in a single unit and restrict direct access of data
→ we can access by using getter and setter methods
Polymorphism :: It means many forms
→ It allows same method name to perform different tasks based on input

we can achieve through
① in-built function ② Method overloading ③ method overriding.

④ method overloading :: By default python won't support method overloading
→ we can overload the methods through variable length positional arguments (*args)

③ method overriding :: If parent class and child class having same method name if we perform inheritance parent class will override child class

Abstraction :: Hiding internal details by showing only required features.
→ we can achieve abstraction by using abstract class and abstract method
→ If we want to create abstract class we need atleast one abstract method
→ Abstraction is not directly supported we need to import a class called ABC and abstract method from abc module

Global
→ It is defined outside the class func
→ It is accessed everywhere
→ It is modify with "global" Keyword

Local
→ It is defined inside the class funct
→ It is accessed inside the funct
→ It is modify inside the funct

function :: function is a reusable block of code in python which takes some arguments and apply those arguments and returns the o/p
→ It is defined with "def" keyword
→ It is mainly used for code reusability

function parameters :: the variable defined inside the funct
function arguments :: the actual value passed when calling the function

Built-in-functions :: these are already defined in python. we use them directly without writing the code

user defined functions :: we can write the code by using "def" Keyword

Positional arguments :: It is mandatory in Python. we need to specify the same position as defined in func def

Keyword arguments :: It is mandatory in python. we need to specify the name of the parameter while calling the function

→ order doesn't matter if we ge use parameter names

default arguments :: It is not mandatory in python. we need to specify the default value in function definition

If no value is passed default is used

variable length arguments ::

① variable length positional arguments ::

→ It allows passing 'n' no. of positional arguments

→ collects them into a tuple

② variable length keyword arguments ::

→ It allows passing 'n' no. of keyword arguments

→ collects them into a dictionary

Error :: from python code that we can't handle

Exception :: Runtime error from python script

→ we can handle by using try & except block

Exception handling :: used to handle exceptions in python

finally :: Irrespective to the result of try and except block finally will get executed

else :: If there is no error in try block else block will get executed

→ If we implement try block, finally (or) except block is mandatory"

→ we use multiple except blocks for one try block

raise :: used to raise our own exceptions

filehandling: used to handle files in python

read :: used to read file contents to a string format

readline :: used to read single line from current cursor position to string format

readlines :: used to read string content to a list format

write :: writes to a file

writelines :: used to write multiple strings to a file at once

lambda :: lambda is an anonymous function in python which is used to defined in a single unit. It takes multiple arguments and single expression

map :: map will map the function to a sequence and returns the map object. we need to convert map object to the required format

filter :: filter is an inbuilt function in python which is used to filter the sequence by applying some function
→ It returns filter object. we need to convert filter object to required format

reduce :: reduce will reduce the sequence to a single element
→ we need to import reduce from python module
@ functools

listcomprehension :: used to optimize the code to reduce no. of lines by writing block of code in a single line

zip :: combines multiple iterables element wise into pairs

enumerate :: used to access index and actual element in a sequence. o/p should be in the form of list of tuples

slicing :: used to extract some portion from original String

for loop

→ It is used when no. of iterations is known
→ It will iterate over the sequence

while loop

→ It is used when condition based repetition is needed
→ It will run as long as condition is true

Range :: Built-in function in python used to generate sequence

loop control statement ::

Break :: It will terminate the loop permanently

continue :: It will skip the current iteration and execute next iteration

Pass :: pass will not make any impact on code. used to define empty loop functions and classes

file handling :: seek( ): To move the cursor position
tell( ): To get the cursor position

regular expressions : used to pattern matching in strings
→ we can use by importing re module

re.search :: search for the keyword in the entire string
re.match :: To match from beginning of the string
re.findall :: To capture all the matched keywords in a list format

. → Any character except newline
^ → start of string, $ → end of string
* → 0 or more repitions , + → 1 or more repition
? → 0 or 1 occurence , {n} → Exactly n repetition
{n, } → n or more repetition , {n,m} → b/w n and m repetition
() → Group , \d → digit , \w → Alpha numeric
\D → Non digit , \W → Non " "

\s → whitespace Group :: we can group the data by using ()
\S → Non " re.sub :: used to find and replace the pattern in string

shallow copy ::
→ creates a new outer list
→ But inner objects are not duplicated they are just refered
→ this means changes to nested elements in the original will also effect the copy

Deep copy ::
→ creates a completely independent copy
→ All nested objects are duplicated
→ changes to the original will not affect the copy

Deep copy takes more memory and slower because they duplicate everything

→ How memory management works in python

In python memory management will be automatically handled by python memory manager. when we assign value to a variable it stores some memory for it. Python uses heap space to store all the variables when we change the value of a variable it marks previous entry as stale entry and it is deleted in the background. this process is known as gc (garbage collection). gc is automatically handled by python memory manager

→ what is python ?

python is a high level interpreter general purpose prgming lang
It has easy syntax simple and easy readability
It is dynamically prgming lang no need to declare a variable
Python interpreter will take care of its runtime
It is platform independent
It has large collection of inbuilt modules

→ How python works ?

python code is executed through the interpreter. interpreter will read the source code and translate it into byte code then python virtual machine will reads the bytecode

→ compiler Vs interpreter

① Translates entire code into machine code before execution	① Translate and execute code line-by-line
② shows all errors after compiling	② shows errors one at a time
Ex :: C, C++, java	Ex :: Python

→ integer :: It represents whole numbers
→ string :: String is a stream of characters enclosed with quotes.
String is immutable datatype once we define String we can't modify String
→ float :: It represents decimal values
→ Boolean :: It represents True/False
→ List ::
collection of elements in python which should be declared in square brackets "[ ]"

list is mutable datatype and ordered data type and it allows duplicate elements
→ it allows indexing, slicing

→ tuple ::
→ tuple collection of elements in python which should be declared in parenthesis "()"
→ tuple is immutable data type once we define it we cannot modify it
→ tuple is faster than list and tuple occupies less space compare to list
→ It allows indexing, slicing
→ we cannot define single element in tuple

→ Set ::
→ collection of elements in python which should be declared in curly brackets "{}"
→ set is mutable datatype it is unordeid and it won't allow duplicate elements
→ It doesn't allow indexing, slicing

→ dictionery ::
→ set of key & value pair in python
→ where keys are immutable values are mutable
→ when we want to map the data b/w two variables we can go for dictionaries
→ It allows duplicate values it won't allow duplicate keys

String methods ::

① lower
② upper
③ swapcase
④ capitilaize
⑤ title
⑥ Index (Keyerror)
⑦ find (-1)
⑧ replace
⑨ rfind
⑩ islower
⑪ isupper
⑫ isalpha
⑬ isspace
⑭ starts with
⑮ ends with
⑯ strip
⑰ rstrip
⑱ lstrip
⑲ isdigit
⑳ isalnum

list methods ::

1. append
2. extend
3. insert
4. remove - remove first occurence in list
5. POP - remove last elem in list. If index given it removes that element
6. copy
7. clear
8. sort
9. reverse
10. count
    - It returns no. of times a specified element is in list

dictionary methods:

1. get - return value if key exist else None
2. setdefault - add value with default value None
3. keys
4. values
5. items
6. POP - (keyerror) if key not exist
7. popitem - By default it remove last key & value pair
8. copy
9. clear
10. update

tuple methods ::

1. count
2. index

set methods ::

1. add
2. update
3. remove - (keyerror)
4. POP
5. copy
6. clear
7. discard - (nothing) (None)
8. union - combine elements in both sets
9. Intersection - common elements in both sets
10. difference - elements in set1 not in set2
11. symmetric differen - return elements that are either set but not both

order of arguments::

→ In functions always non-positional args followed by positional args

→ So, order must be ::

pos args > default args > *args > ** kwargs

in-built ::

max(), min(), len(), sum(), sorted(), type(), zip(), enumerate(), range()

return :: It will stores the value and it will send the value back to the caller

→ once the return is called func will be terminated immediately

→ If no return is used in a func func will return None as default value (In python default return value will be "None")

---

madhavi (capgemini)

→ Self

→ Why we use storage devices?

→ What is backup recovery

→ what is SAN, NAS

→ what is snapshot and how it is used?

→ what is thin provisioning

→ RAID 9 levels? RAID 5?

→ Snapshot Vs backup?

→ Can u use any [illegible]

Operators ::

① Arithematic operator ::

Addition (+), Subtraction (-), multiplication (*), division (/), floor division (//), modulus (%), Exponential (**)

② comparison operator ::

equal to (==), not equal to (!=), greater than (>), less than (<), greater than or equal to (>=), less than or equal to (<=)

③ Logical operator ::

and - True if both true
or - True if one true
not - Inverts the result

④ Assignment operator ::

Assign (=), add & assign (+=), subtract & assign (-=), multiply & assign (*=), divide and assign (/=)

⑤ membership operator ::

in - present, not in - not present

⑥ Identity operator ::

is - same object, is not - not the same object

⑦ Bitwise operator ::

& - AND, ^ - XOR, ~ - NOT, << - left shift, >> - right shift

⟹ Diff b/w "is" and "=="

"is" - It will compare the memory location of two variables/ object

"==" - It will compare the actual value of two variables

→ memory location same in "immutable" diff in "mutable"

⟹ Name mangling :: used to access private variables outside the class

⟹ __new__ :: It is a special method in python used to create new objects and __new__ will be called before creating an object

→ we can control the no. of objects created for a class from __new__ method